SmartLink Git Quiz: Question Bank

Oct 23, 2026 · @dhinakaran

Private: contains the answer key. Do not share with participants.

15 main questions (5 easy, 5 medium, 5 hard) and 5 reserve questions for the sudden-death tie-break. The quiz server shuffles the option order per student, so the letters below are only for this review.

How the answers were verified

Every correct answer, and every wrong option that makes a claim about how Git behaves, was run in a throwaway repository on Git 2.43.0: 66 checks, all passed. Checks that need a remote used a local bare repository.

Not run: pure wording distractors (for example "They do exactly the same thing" in Q9), `git log` and `git show` in Q2, `git blame`, `git grep` and `git log` in R3, and `git commit -m "oops"` in R2.

Before you use it

- Take all 15 yourself with a 20-second timer, and relabel any question you cannot answer in time.
- Check each question against what your webinar speaker actually teaches.
- Ask a co-organizer to read every question once.
- Difficulty labels are estimates until you have real answers to calibrate them.

Easy

Q1 · Basics. Which command turns the current folder into a new Git repository?

- A) `git start`
- B) `git init`
- C) `git new`
- D) `git create`

Answer: B. `git init` creates the hidden .git folder that stores the repository. The other three are not Git commands.

Q2 · Basics. Which command shows which files are modified, staged, or untracked?

- A) `git log`
- B) `git diff --stat`
- C) `git status`
- D) `git show`

Answer: C. `git status` lists staged, modified and untracked files. `git diff --stat` ignores untracked files, `git log` shows history, and `git show` displays a single commit.

Q3 · Branching. You want to create a new branch called feature and switch to it in one command. Which one?

- A) `git branch feature`
- B) `git checkout feature`
- C) `git switch feature`
- D) `git switch -c feature`

Answer: D. The -c flag creates the branch and switches to it. `git branch` only creates it, and `git checkout` or `git switch` without -c fail when the branch does not exist yet.

Q4 · Remote. Which command copies an existing remote repository, with its full history, to your computer?

- A) `git clone`
- B) `git copy`
- C) `git fork`
- D) `git pull`

Answer: A. `git clone` downloads the whole repository and sets up the origin remote. `git pull` only works inside an existing repository, and "fork" is a GitHub feature, not a Git command.

Q5 · Undo. You staged notes.txt by mistake. You want to unstage it but keep your edits in the file. Which command?

- A) `git reset --hard`
- B) `git stash`
- C) `git restore --staged notes.txt`

- D) `git restore notes.txt`

Answer: C. `git restore --staged` moves the file out of the staging area and leaves your edits in place. `git restore` without --staged and `git reset --hard` throw the edits away, and `git stash` removes them from the file.

Medium

Q6 · Basics. debug.log is already tracked by Git. You now add `*.log` to .gitignore. What happens to debug.log?

- A) Git deletes it from the repository
- B) Git keeps tracking it and still reports its changes
- C) Git automatically stops tracking it
- D) Git refuses to commit any further changes to it

Answer: B. .gitignore only affects untracked files. A tracked file stays tracked until you run `git rm --cached` on it.

Q7 · Branching. main has no new commits since you created feature from it. On main you run `git merge feature`. What happens?

- A) A merge commit is always created
- B) Git reports a merge conflict
- C) main moves forward to feature with no merge commit (fast-forward)
- D) Nothing happens unless you add --force

Answer: C. When main has not diverged, Git simply moves the main pointer forward. No merge commit is needed unless you pass --no-ff.

Q8 · Branching. Branch old-login has commits that were never merged anywhere, and you want it gone. Which command deletes it?

- A) `git branch -d old-login`
- B) `git branch --remove old-login`
- C) `git switch -d old-login`
- D) `git branch -D old-login`

Answer: D. -d is the safe delete and refuses when commits are unmerged. -D forces the deletion. `git switch -d` only detaches HEAD, and --remove is not a valid option.

Q9 · Remote. What is the difference between `git fetch` and `git pull`?

- A) fetch merges new commits into your branch; pull only downloads them
- B) fetch downloads new commits without changing your current branch; pull downloads and then merges them
- C) They do exactly the same thing
- D) fetch uploads your commits; pull downloads them

Answer: B. `git fetch` only updates your remote-tracking branches. `git pull` is a fetch followed by a merge (or a rebase if configured).

Q10 · Undo. A bad commit was already pushed to the shared main branch. What is the safest way to undo it without rewriting history?

- A) `git reset --hard HEAD~1`, then `git push --force`
- B) `git commit --amend`
- C) `git revert HEAD`
- D) `git rebase -i` and drop the commit

Answer: C. `git revert` adds a new commit that cancels the bad one and keeps history intact. The other options rewrite history, which causes problems for teammates who already pulled.

Hard

Q11 · Undo. You made your last 3 commits by mistake. Undo them, but keep all their changes staged and ready to recommit. Which command?

- A) `git reset --hard HEAD~3`
- B) `git reset --mixed HEAD~3`
- C) `git reset --soft HEAD~2`
- D) `git reset --soft HEAD~3`

Answer: D. --soft moves the branch back and keeps the changes staged. --mixed leaves them unstaged, --hard discards them, and HEAD~2 only undoes two commits.

Q12 · Undo. You ran `git reset --hard HEAD~2` by mistake and lost two commits. Nothing was pushed or stashed. Which command helps you find them?

- A) `git log --all`
- B) `git stash list`
- C) `git status`

- D) `git reflog`

Answer: D. `git reflog` records where HEAD has been, so the lost commits appear there. You can recover with `git reset --hard <hash>`. `git log --all` does not list commits no branch points to.

Q13 · Merging. `git merge feature` stops with a conflict in app.py. Which sequence correctly finishes the merge?

- A) `git commit` immediately
- B) `git reset --hard`
- C) Edit app.py to remove the conflict markers, `git add app.py`, then `git commit`
- D) `git rebase --continue`

Answer: C. Resolve the file, stage it to mark it resolved, then commit to complete the merge. Committing right away fails, `git reset --hard` abandons the merge, and there is no rebase to continue.

Q14 · Remote. Your push is rejected because a teammate pushed first. You want to keep history linear (no merge commit) without overwriting their work. What do you run?

- A) `git push --force`
- B) `git pull`, then `git push`
- C) `git fetch`, then `git push`
- D) `git pull --rebase`, then `git push`

Answer: D. `git pull --rebase` replays your commits on top of the teammate's work, so history stays linear and the next push is accepted. A plain pull creates a merge commit, fetch alone does not integrate anything, and --force would erase their commit.

Q15 · Remote. You rebased feature onto main after pushing it, and `git push` is now rejected. What is the safest way to update the remote branch?

- A) `git push --force`
- B) `git pull`, then `git push`
- C) `git push --no-verify`
- D) `git push --force-with-lease`

Answer: D. Rebasing rewrites commit hashes, so a force push is needed. --force-with-lease refuses if someone else pushed in the meantime, while --force overwrites blindly. Pulling merges the old and new copies and duplicates commits, and --no-verify only skips hooks.

Reserve questions (sudden-death)

Keep these hidden from the main quiz. In a tie-break, use one question, with the same version for every student involved.

R1 · Undo. You have a modified tracked file and a brand-new untracked file. Which command shelves both so you can switch tasks?

- A) `git stash`
- B) `git stash -u`
- C) `git stash pop`
- D) `git stash apply`

Answer: B. -u (include untracked) stashes new files too. Plain `git stash` leaves untracked files behind, and pop and apply bring a stash back rather than creating one.

R2 · Undo. You just committed but forgot to include a new file, forgotten.txt. How do you add it to that same commit without changing the message?

- A) `git commit --amend --no-edit`
- B) `git add forgotten.txt`, then `git commit --amend --no-edit`
- C) `git commit -m "oops"`
- D) `git revert HEAD`

Answer: B. Stage the file first, then amend. Amending without staging does not include the new file, and a second commit would not be the same commit.

R3 · Debugging. Which command starts a binary search through your history to find the commit that introduced a bug?

- A) `git blame`
- B) `git bisect start`
- C) `git grep`
- D) `git log --oneline`

Answer: B. `git bisect` marks a known good and a known bad commit, then checks out commits in between so you find the culprit in a few steps.

R4 · Merging. You want to bring the changes of a 2-commit feature branch into main as staged changes, with no commit made yet, so you can commit them as one. Which command?

- A) `git merge --no-ff feature`

- B) `git cherry-pick feature`
- C) `git merge --squash feature`
- D) `git merge feature`

Answer: C. --squash stages the combined changes and leaves the commit to you. --no-ff creates a merge commit, cherry-pick applies only the tip commit, and a plain merge may fast-forward and keep both commits.

R5 · Remote. You created a tag v1.0 locally. Which command shares just that tag with the remote?

- A) `git push`
- B) `git tag --push v1.0`
- C) `git push origin v1.0`
- D) `git commit --tag v1.0`

Answer: C. Tags are not sent by a normal push. Name the tag explicitly: `git push origin v1.0`.

Answer key

Q	Answer	Q	Answer	Q	Answer
Q1	B	Q6	B	Q11	D
Q2	C	Q7	C	Q12	D
Q3	D	Q8	D	Q13	C
Q4	A	Q9	B	Q14	D
Q5	C	Q10	C	Q15	D

Reserve: R1 B · R2 B · R3 B · R4 C · R5 C